

AI-Assisted Lending Intelligence Module (ALIM)

Technical Design Document (TDD)

1. Document Control

Attribute	Value
Project Name	AI-Assisted Lending Intelligence Module (ALIM)
Document Type	Technical Design Document
Version	1.0
Prepared By	Solution Architecture Team
Intended Audience	Architects, Developers, DevOps, Security, Platform Teams
Repository	backend/ monorepo
Architecture Style	Event-Driven Microservices
Technology Stack	Java 17, Spring Boot 3.x, Spring AI
Deployment Target	Kubernetes / Cloud-native Infrastructure

2. Executive Summary

The AI-Assisted Lending Intelligence Module (ALIM) is a modular enterprise intelligence layer integrated into the existing lending ecosystem to provide contextual AI-assisted capabilities for underwriting, policy interpretation, document understanding, and workflow intelligence.

The system is designed as a scalable, event-driven, cloud-native architecture supporting:

- semantic document retrieval
- AI-assisted underwriting
- explainable AI responses
- contextual policy intelligence
- human-in-loop workflows
- auditability and compliance

The platform integrates deterministic business rule enforcement with self-hosted AI-assisted contextual reasoning while ensuring enterprise confidentiality, governance, observability, and operational resilience.

All AI inference workloads shall execute within organization-controlled infrastructure using approved offline or privately hosted models. No sensitive lending or customer data shall be transmitted to external frontier model providers.

3. Background & Business Context

Enterprise lending operations currently face:

- high manual underwriting effort
- fragmented document understanding
- operational delays
- manual policy interpretation
- inconsistent contextual analysis
- limited AI governance capabilities

The organization requires a controlled AI integration layer that:

- augments existing workflows
- preserves deterministic governance
- enables contextual retrieval
- maintains explainability
- supports enterprise scalability

The AI layer shall operate as an advisory intelligence component and shall not independently approve or reject lending applications.

Due to regulatory, confidentiality, and data residency requirements, the organization mandates that all AI inference and embedding generation capabilities operate within controlled enterprise environments without reliance on public frontier AI APIs.

4. Architecture Goals & Principles

4.1 Architecture Goals

The target architecture must:

- support modular extensibility
- support AI-provider abstraction
- support event-driven asynchronous workflows
- support explainable AI interactions
- maintain auditability
- support scalable semantic retrieval
- isolate domain responsibilities
- support enterprise observability

4.2 Architecture Principles

Principle	Description
Domain-Driven Design	Business domains isolated by bounded contexts
AI as Advisory Layer	AI recommendations remain non-authoritative
Explainability First	Every AI response must be traceable
Event-Driven Processing	Async orchestration via domain events
Provider Abstraction	AI providers abstracted behind interfaces
Stateless Services	Services horizontally scalable
Human-in-Loop	Final decisions remain human-controlled
Security by Design	Security integrated into all layers
Enterprise AI Isolation	All AI workloads must execute within organization-controlled environments

5. Constraints & Assumptions

5.1 Constraints

Constraint	Description
Regulatory Compliance	Lending operations are regulated
Explainability	AI outputs must be explainable
Existing Platform Dependency	Existing lending platform remains source of truth

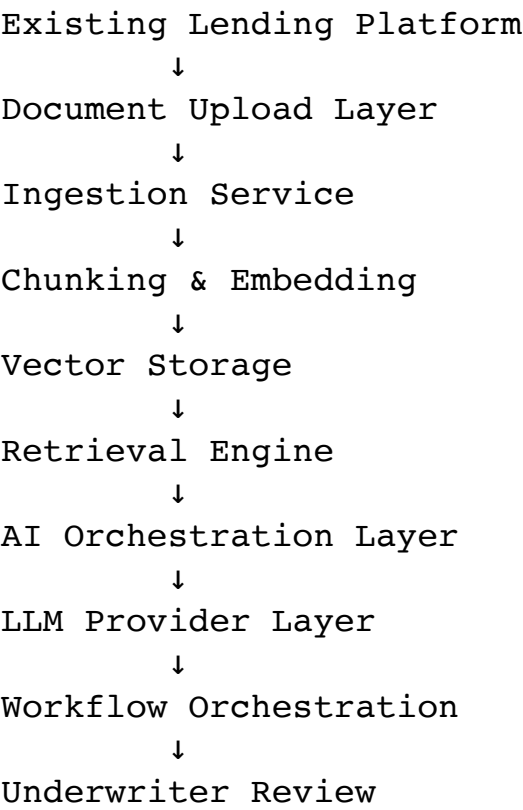
Cloud Restrictions	Infrastructure may require hybrid deployment
AI Latency	External LLM providers introduce latency
No External AI APIs	Sensitive lending data cannot be shared with public AI providers
Air-Gapped Compatibility	Architecture should support isolated deployment environments

5.2 Assumptions

Assumption	Description
Existing Authentication	Existing IAM solution available
Existing Loan Lifecycle	Loan lifecycle already implemented
Existing Document Upload	File upload infrastructure available
Loan Identifiers	loanId globally unique
AI Connectivity	External AI endpoints accessible

6. System Overview

6.1 High-Level Architecture



6.2 Existing Monorepo Structure

```
backend/  
├── common/  
├── ai-core/  
├── ingestion/  
├── lending/  
└── app/
```

6.3 Proposed Expanded Monorepo Structure

```
backend/  
├── common/  
├── ai-core/  
├── ingestion/  
├── lending/  
├── workflow/  
├── rule-engine/  
├── audit/  
├── policy/  
├── orchestration/  
├── document-intelligence/  
├── integration/  
└── app/
```

7. Architecture Decisions (ADRs)

ADR-001: Event-Driven Architecture

Decision

Adopt Kafka-based event-driven asynchronous orchestration.

Rationale

- loose coupling
- async scalability
- operational resiliency
- workflow extensibility

ADR-002: AI Provider Abstraction

Decision

All AI providers shall be abstracted using interfaces.

Rationale

- support offline/self-hosted models
- provider portability
- controlled enterprise deployment
- model governance
- future on-prem AI evolution

ADR-003: Human-in-Loop Governance

Decision

AI outputs remain advisory only.

Rationale

- regulatory compliance
- operational governance
- explainability
- risk mitigation

ADR-004: Hybrid Rule + AI Validation

Decision

Deterministic validations shall remain rule-engine driven.

Rationale

- explainability

- deterministic outcomes
- regulatory clarity
- operational reliability

ADR-005: Vector Search Architecture

Decision

Adopt pluggable vector store abstraction.

Rationale

- future scalability
- vendor flexibility
- storage portability

ADR-006: Offline AI Deployment Strategy

Decision

AI inference and embedding generation shall use organization-approved offline or privately hosted models.

Rationale

- regulatory compliance
- enterprise confidentiality
- PII protection
- data residency requirements
- operational control
- vendor independence

8. Component Design

8.1 common/

Responsibility

Shared contracts, enums, exceptions, utilities.

Key Components

- BaseException
- DocumentStatus
- shared DTOs
- common utilities

8.2 ai-core/

Responsibility

Core AI abstractions and orchestration.

embedding/

Responsibilities

- embedding generation
- provider abstraction
- request normalization

Key Interfaces

EmbeddingProvider

EmbeddingRequest

EmbeddingResult

llm/

Responsibilities

- chat completion abstraction
- response normalization
- provider-specific integration

Key Interfaces

LlmProvider
LlmRequest
LlmResponse

rag/

Responsibilities

- retrieval orchestration
- context assembly
- response generation

retrieval/

Responsibilities

- semantic retrieval
- metadata filtering
- ranking

vectorstore/

Responsibilities

- vector persistence abstraction
- semantic search
- provider portability

8.3 ingestion/

Responsibility

Document ingestion pipeline.

Processing Flow

Document Upload

- Parsing
- Chunking
- Embedding
- Vector Persistence
- Event Publication

8.4 workflow/

Responsibility

Workflow orchestration.

Technology

Camunda / Temporal.

Responsibilities

- AI review lifecycle
- manual review orchestration
- workflow transitions

8.5 rule-engine/

Responsibility

Deterministic validations.

Technology

Drools / DMN.

Responsibilities

- mandatory document validation
- policy thresholds
- eligibility rules

8.6 audit/

Responsibility

AI explainability and traceability.

Responsibilities

- prompt persistence
- AI response logging
- retrieved chunk logging
- model metadata tracking

8.7 orchestration/

Responsibility

Coordinate AI + workflow + rules.

Responsibilities

- orchestration pipelines
- confidence routing
- failure coordination
- retry orchestration

9. Data Architecture

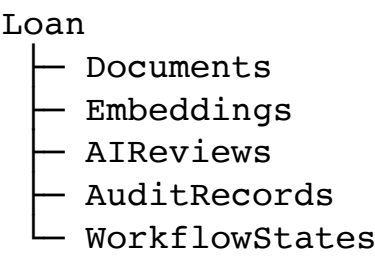
9.1 Data Categories

Category	Description
Operational Data	Loan workflow data
AI Metadata	Prompts, responses, embeddings
Vector Data	Semantic embeddings
Audit Data	Explainability records
Policy Data	Lending policies

9.2 Storage Architecture

Data Type	Storage
Transactional Data	PostgreSQL
Vector Embeddings	ChromaDB / PGVector
Audit Logs	PostgreSQL / Elasticsearch
Metrics	Prometheus
Distributed Traces	Tempo / Jaeger

9.3 Entity Relationships



9.4 Metadata Design

Each vector document shall contain:

- loanId
- customerId
- documentType
- sourceDocumentId
- ingestionTimestamp
- chunkSequence
- policyVersion

10. Integration Design

10.1 Internal Integrations

Integration	Purpose
Lending Service	Loan lifecycle retrieval
Workflow Engine	Process orchestration
Rule Engine	Deterministic validations
Audit Service	Explainability persistence

10.2 External Integrations

Integration	Purpose
Self-Hosted LLM Runtime	AI inference and completions
Self-Hosted Embedding Runtime	Embedding generation
OCR Provider	Document OCR
Monitoring Platform	Metrics aggregation

10.3 Event Integration

Domain Events

LoanSubmitted
 DocumentUploaded
 DocumentIngested
 EmbeddingGenerated
 AiReviewRequested
 AiReviewCompleted
 ManualReviewRequired
 UnderwriterApproved
 LoanRejected

11. Security Architecture

11.1 Authentication

The system shall integrate with enterprise IAM.

Supported:

- JWT
- OAuth2

- SSO integration

11.2 Authorization

RBAC shall be enforced.

Role	Access
UNDERWRITER	AI review access
RISK_ANALYST	Risk observation access
COMPLIANCE_USER	Audit access
ADMIN	Administrative configuration

11.3 Data Security

The system shall:

- encrypt data at rest
- encrypt data in transit
- support PII masking
- support secure secret management

11.4 AI Security

The system shall support:

- prompt sanitization
- prompt injection mitigation
- output validation
- provider access controls

The AI platform shall support isolated model execution within organization-controlled infrastructure. No prompts, embeddings, customer documents, or inference payloads shall leave the enterprise network boundary.

12. Infrastructure & Deployment

12.1 Deployment Architecture

Kubernetes Cluster

- API Gateway
- Lending Services
- AI Core Services
- Workflow Services
- Kafka Cluster
- PostgreSQL
- Vector Database
- Observability Stack

12.2 Containerization

All services shall:

- be containerized
- support immutable deployments
- support rolling updates

12.3 CI/CD

Pipeline stages:

Build

- Unit Tests
- Static Analysis
- Security Scanning
- Container Build
- Integration Tests
- Deployment

12.4 AI Infrastructure

AI inference workloads shall be deployed using self-hosted model runtimes within enterprise infrastructure.

Supported deployment patterns:

- on-premise GPU clusters
- private cloud AI infrastructure
- air-gapped deployment environments

- Kubernetes-based model serving

Model serving frameworks may include:

- Ollama
- vLLM
- Text Generation Inference (TGI)
- llama.cpp

13. Scalability & Performance

13.1 Scalability Requirements

The system shall support:

- horizontal scaling
- asynchronous ingestion
- distributed vector search
- stateless service instances

13.2 Performance Targets

Operation	SLA
Document ingestion	< 5 sec
Retrieval search	< 2 sec
AI summarization	< 8 sec
Policy Q&A	< 5 sec

13.3 Performance Strategies

- async processing
- retrieval caching
- embedding reuse
- batching

- event buffering
- distributed consumers

14. Reliability & Fault Tolerance

14.1 Reliability Requirements

The target availability shall be 99.5%.

14.2 Fault Tolerance Strategies

Strategy	Description
Retry Policies	AI invocation retries
Dead Letter Queues	Failed event handling
Circuit Breakers	External dependency protection
Idempotency	Safe retry handling
Graceful Degradation	Partial feature availability

14.3 AI Failure Handling

If AI provider unavailable:

- workflow continues
- review marked pending
- manual review initiated
- alert generated

15. Observability

15.1 Metrics

The system shall capture:

- API latency

- token usage
- retrieval latency
- embedding latency
- workflow duration
- event lag
- ingestion failures

15.2 Logging

Structured centralized logging shall be implemented.

15.3 Distributed Tracing

OpenTelemetry tracing shall support:

- request tracing
- event tracing
- AI invocation tracing
- workflow tracing

15.4 Dashboards

Operational dashboards shall include:

- AI usage metrics
- workflow health
- ingestion throughput
- vector search latency
- provider availability

16. Migration Plan

16.1 Migration Strategy

The module shall be integrated incrementally.

Phase 1

- introduce ingestion
- introduce vector storage
- enable policy retrieval

Phase 2

- integrate workflow orchestration
- integrate audit services
- enable AI review assistance

Phase 3

- advanced orchestration
- MCP integrations
- confidence routing

17. Risk Register

Risk	Impact	Mitigation
AI hallucination	Incorrect observations	Evidence-backed retrieval
High latency	Operational delays	Async workflows
Vendor dependency	Lock-in risk	Provider abstraction
Compliance concerns	Regulatory exposure	Audit logging
Cost escalation	Operational overhead	Token monitoring
Prompt injection	Security risk	Prompt sanitization
Confidential data leakage	Regulatory exposure	Offline/self-hosted AI deployment

Model governance drift	Compliance issues	Centralized model approval lifecycle
------------------------	-------------------	--------------------------------------

18. Open Questions

Question	Owner
Final vector database selection	Architecture Team
OCR provider strategy	Platform Team
Multi-provider AI routing	AI Platform Team
Long-term retention policy	Compliance Team
Policy version governance	Product Team

19. Glossary

Term	Meaning
RAG	Retrieval-Augmented Generation
LLM	Large Language Model
Embedding	Numerical semantic representation
Vector Store	Semantic search database
MCP	Model Context Protocol
SLA	Service Level Agreement
RBAC	Role-Based Access Control
OCR	Optical Character Recognition

20. Appendix

20.1 Recommended Technology Stack

Area	Technology
Language	Java 17
Framework	Spring Boot 3.x
AI Framework	Spring AI
Offline LLM Runtime	Ollama / vLLM / TGI
Embedding Runtime	BGE / E5 / Instructor Models
Event Streaming	Kafka

Workflow	Camunda / Temporal
Rule Engine	Drools
Database	PostgreSQL
Vector Store	ChromaDB / PGVector
Security	Spring Security
Observability	Prometheus + Grafana

20.2 Sample Event Flow

```
LoanSubmitted
  ↓
DocumentUploaded
  ↓
DocumentIngested
  ↓
EmbeddingGenerated
  ↓
AiReviewRequested
  ↓
AiReviewCompleted
  ↓
ManualReviewRequired
  ↓
UnderwriterApproved
```

20.3 Future Expansion Areas

- bureau integrations
- fraud intelligence
- multi-agent orchestration
- operational copilots
- AI governance dashboards
- multilingual policy intelligence
- enterprise MCP ecosystem